Snekboros: Vision World Models for Snake Playing

1006652 Lee Jun Hui Ryan, 1006895 Wang Yihe, 1007036 Celest Teng Roh Yee, 1007485 Gregory Lim Eu Rhen

1 Introduction

Arguably, AI systems today do not possess "human-level intelligence" for many important real-world tasks. Driving is a common example: humans can learn the basics in days, yet level-five autonomous driving remains out of reach even with millions of hours of data. World models have been proposed as a promising approach to human-level AI. Broadly, a world model learns a representation of its environment, and how to evolve that representation through action, thus "learning the consequences of their actions". To reach a particular goal at inference, a world model can be used to plan action sequences optimising a particular objective. In recent years, world models have demonstrated promise in learning tasks ranging from robotic manipulation to video game playing on relatively low sample counts. For this reason, we believe world models present an exciting opportunity for innovation in AI.

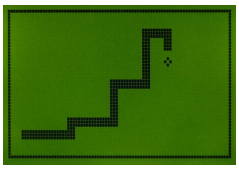


Figure 1: Snake on a Nokia 6110 phone.

Snake is a classic action video game where the player controls the end of a growing line segment on a planar region, steering it left, right, up, and down to consume food without colliding with walls or snake segments (Wikipedia contributors, 2026). Although Snake has simple rules, games often require long-horizon planning, as a single wrong move early on can be fatal. In 2018, OpenAI released a set of research requests, one of which asked researchers to train a machine learning model to beat the classic Snake game. This request serves as the impetus for our research project (Sutskever et al., 2018).[1]

In this paper, we investigate the use of vision world models—world models with visual input—for the game of Snake. Our contributions are threefold. First, we formulate Snake as a vision-based problem. Second, we train and test world models for Snake playing. Third, we compare and interpret the pros and cons of various approaches. Our code repository can be found at github.com/gregorylimeurhen/snekboros. Snekboros aims to contribute most directly to SDG 9 (Industry, Innovation, and Infrastructure) by advancing world model research.

[1] We believe Snake to be an unusually clean testbed for world modelling as it admits a trivial solution—moving along a Hamiltonian cycle—on grids with even dimension.

2 Existing Work

Model-based learning, an approach in which an agent learns an internal model of environment dynamics, has a long history in robotics (Sutton, 1991; Holkar et al., 2011; Williams et al., 2017). By learning to predict how an environment evolves, world models enable agents to reason about possible futures, in a process analogous to human imagination.

Generative world models learn to predict environment dynamics in pixel space, acting as learned simulators that generate future observations from a history of past states and actions. Historically, these models have been fairly successful in various games (Hafner et al., 2025; Micheli et al., 2023; Decart et al., 2024; Alonso et al., 2024). However, these models predict in high-dimensional pixel space, which can be computationally expensive (Ko et al., 2023). Additionally, these models risk wasting capacity on learning task-irrelevant noise as details in pixel space are not always predictable. These models also sometimes use task-specific state-based objectives, which hinder adaptation to new tasks.

The Joint Embedding Predictive Architecture (JEPA) has emerged as a promising way for world models to learn to predict environment dynamics in representation space (LeCun, 2022). JEPA has prominently been applied for self-supervised representation learning by predicting masked image patches in representation space (Assran et al., 2023; Bardes et al., 2024; Assran et al., 2025). However, JEPA admits a trivial solution that leads to representation collapse, as the encoder can learn to map all observations to a single representation. Early approaches tried to address this collapse by devising a gradient-free "target encoder" that tracks an exponential moving average of the (actual) encoder. Other approaches, such as DINO-WM, simply use frozen pretrained vision encoders (Zhou et al., 2025). Although this prevents representation collapse and can make training more straightforward, this could closely tie a system to its encoder's input resolution, and relies on how well the pretrained weights match the (actual) task.

Model predictive control (MPC) predicts and evaluates candidate action sequences online, and has demonstrated its ability to ground real-time dynamical models at the cost of increased computation. However, many works only focus on standard robotics tasks with explicit goal states (e.g. Maze, Wall, PushT, Reach, etc.) used as input at test time. (Fu et al., 2021; Tassa et al., 2018). Our work explores MPC-style approaches that have no such input at test time.

3 Methodology

3.1 Data

Let $L \in 2\mathbb{N}_{\geq 1}$ be a grid extent, and $U \in \mathbb{N}_{\geq 1}$ be an upsampling factor, and $D \in \mathbb{N}_{\geq 1}$ be a time horizon, and $\mathcal{G} = [L]^2$ be a grid, and $\preceq$ be the row major order of $\mathcal{G}$, and $\mathcal{X} = [0,1]^{LU \times LU \times 3}$ be an image space.

Definition 1 (Action Space)**.**

$$\mathcal{A} = \{-1, 0, 1\}$$

Definition 2 (Movement Space)**.**

$$\mathcal{M} = \{(0,1), (0,-1), (-1,0), (1,0)\}$$

Definition 3 (Rotation Function)**.**

$$R : \mathbb{Z}^2 \to \mathbb{Z}^2$$
$$R(x, y) = (y, -x)$$

The action space contains actions for "left", "right", and "straight". The movement space contains movements for "up", "down", "left", and "right". The rotation function rotates a grid position clockwise by 90°.

Algorithm 1 SIMULATE(δ)

Require: $\delta : \mathbb{N} \to \mathcal{A}$
  $\boldsymbol{s} \sim \mathcal{U}(\{\boldsymbol{s} : [2] \to \mathcal{G} \mid s_2 - s_1 \in \mathcal{M}\})$
  $E \leftarrow \mathcal{G} \setminus \{s_i \mid i \in [|\boldsymbol{s}|]\}$
  $f \leftarrow \min_{\preceq} E$
  $t \leftarrow 0$
  while $\boldsymbol{s} \neq \perp$ **do**
    $i_f \leftarrow 0$
    $a \leftarrow \delta(t)$
    $h \leftarrow s_1 + R^a(s_1 - s_2)$
    if $h = f$ **then**
      $\boldsymbol{s} \leftarrow h \parallel (s_i)_{i=1}^{|\boldsymbol{s}|}$
      $i_f \leftarrow 1$
    else
      $\boldsymbol{s} \leftarrow h \parallel (s_i)_{i=1}^{|\boldsymbol{s}|-1}$
    $B \leftarrow \{s_i \mid i \in [2..|\boldsymbol{s}|]\}$
    if $s_1 \notin \mathcal{G} \setminus B$ **then**
      $\boldsymbol{s} \leftarrow \perp$
    if $\boldsymbol{s} \neq \perp \wedge i_f = 1$ **then**
      $E \leftarrow \mathcal{G} \setminus \{s_i \mid i \in [|\boldsymbol{s}|]\}$
      if $E = \varnothing$ **then**
        $f \leftarrow \perp$
      else
        $f \leftarrow \min_{\preceq} E$
    $t \leftarrow t + 1$

The game starts by spawning a length-2 snake in random adjacent cells. At each tick, the snake takes an action to move its segments, and the food spawns in the first cell under row-major order if there is an unoccupied cell and every cell is unoccupied by food. The snake dies if its first segment moves out of the grid or into a cell occupied by another segment. The snake grows by 1 (from its last segment) if its first segment moves into a cell occupied by food. The snake wins if its segments cover the grid.

For experimental reproducibility, we use a fixed, global seed for each pseudorandomly generated quantity. We set $L = 4$ and $U = 1$ to keep computation tractable while keeping planning non-trivial.

To avoid manually playing the game, we generate episodes using a perturbed Hamiltonian policy with first-action off-policy branching on each game tick for coverage of a large and diverse number of plausible game states (Tapsell, 2015). For each trajectory, we store label

$$(\mathfrak{S}, (\mathfrak{C}_t)_{t=1}^{D})$$

where survival label $\mathfrak{S}$ is 1 if the snake survived the trajectory (and 0 otherwise), and for each $t \in [D]$, consumption label $\mathfrak{C}_t$ is 1 if the snake consumed at game tick t (and 0 otherwise).

Each unoccupied cell is coloured black. Each cell occupied by a snake segment $s_i \in \boldsymbol{s}$ is coloured blue with brightness $1 - (i-1)/L^2$ for each $i \in [|\boldsymbol{s}|]$. Each cell occupied by food is coloured green. When the snake dies (resp. wins), each cell becomes and remains black (resp. white) for the remainder of the trajectory. The cells are coloured in this non-overlapping way to ensure injectivity for the game to be Markovian after snake spawning. The grid is rendered at arbitrary scale by rasterisation.

For each starting position, we generate 1 episode, and the data is split by episode into training, validation, and testing sets in a ratio of 8-1-1.

3.2 Architecture

Let $\mathcal{H} \in \{\mathbb{R}^H \mid H \in \mathbb{N}_{\geq 1}\}$ be a hidden space, and $\tau \in [0,1]$ be a survival threshold, and $\boldsymbol{\gamma} \in [0,1]^D$ be a discount.

Definition 4 (Encoder)**.**

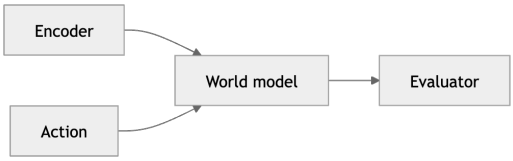
$$\phi : \mathcal{X} \to \mathcal{H}$$

Definition 5 (World Model)**.**

$$\mathscr{F} : \mathcal{H} \times \mathcal{A} \to \mathcal{H}$$

Definition 6 (Evaluator)**.**

$$\mathscr{V}_S : \mathcal{H}^D \to [0,1]$$
$$\mathscr{V}_C : \mathcal{H}^D \to [0,1]^D$$
$$\mathscr{V} : \mathcal{H}^D \to \mathbb{R}_{\geq 0}$$
$$\mathscr{V}(\boldsymbol{h}) = \mathbf{1}_{\mathscr{V}_S(\boldsymbol{h}) \geq \tau} \boldsymbol{\gamma}^\top \mathscr{V}_C(\boldsymbol{h})$$



The system consists of three components: an encoder, a world model, and an evaluator. The encoder maps an image $x \in \mathcal{X}$ into a representation $h \in \mathcal{H}$ that aims to capture abstract information needed for effective downstream prediction by eliminating unnecessary detail. The world model, conditioned on an action $a \in \mathcal{A}$, maps a representation $h \in \mathcal{H}$ to a next representation $h' \in \mathcal{H}$ that aims to represent the state of the game at the next game tick. The evaluator maps a representation sequence $\boldsymbol{h} \in \mathcal{H}^D$ to a number $r \in \mathbb{R}_{\geq 0}$ that aims to measure how good (or bad) the state corresponding to that representation is. The evaluator has a survival head and a consumption head. The survival (resp. consumption) head learns to predict the probability of (i.e. approximate the) survival (resp. consumption) labels.

Our components use convolutional and gated recurrent unit (GRU) architectures. Our encoder uses a convolutional architecture, while our world model and evaluator use convolutional GRU architectures.

We use both one-step teacher-forced and multi-step free-running loss terms to supervise individual transitions and enforce consistency on rollouts. The world model uses mean squared error (MSE) loss with stop gradients on encoded successors. The evaluator uses binary cross-entropy (BCE) loss on survival and consumption labels.

There is a trivial solution where the encoder maps all observations to a single representation. To avoid this "representation collapse", we apply sketched isotropic Gaussian regularisation (SIGReg) for representations to follow an approximately isotropic Gaussian distribution (Balestriero and LeCun, 2025). We monitor training for representation collapse by tracking the effective rank of the centred and flattened representations.

We train the system in phases for numerical stability. The first 20% of epochs only update encoder weights. The next 20% of epochs only update world model weights. The last 60% of epochs update encoder, world model, and evaluator weights.

Inference returns the first element of

$$\arg \max_{\boldsymbol{a} \in \mathcal{A}^D_{\checkmark}(x)} \mathscr{V}\Big(\big[\big(\bigcirc_{i=1}^{t} \mathscr{F}_{a_i} \big)(\phi(x)) \big]_{t=1}^{D} \Big)$$

where $x \in \mathcal{X}$ is an image, and $\mathcal{A}^D_{\checkmark}(x) \subseteq \mathcal{A}^D$ is the space of legal length-D action sequences from the game state implied by x. As enumerating over all action sequences is often intractable, we sample using a (first-action-)stratified random method. We set the action sequence sample count to 1024 to balance computational tractability with planning quality. Ties are broken randomly over first actions of optimal legal sequences. We set $\tau = 0.25$ to balance snake aggression with conservativeness. We set $D = L^2 - 1$ as it is the maximum length of a simple path on $\mathcal{G}$. We set $\boldsymbol{\gamma} = ((1 - 1/D)^{t-1})_{t=1}^{D}$ for food consumption to matter more earlier.

4 Experimentation

In a game, a snake can consume a maximum of $L^2 - 2$ foods (in total). Thus, we define the completion rate of a game as the fraction of foods consumed by a snake in that game. We make completion rate our scoring metric since in a game, a snake wins by consuming the maximum number of foods. To prevent a snake from prolonging a game indefinitely by looping, we enforce a maximum of 1000 game ticks, beyond which the snake is considered dead.

Survival threshold	**Score**
0.00	0.86
0.25 (baseline)	0.89
0.50	0.86

Table 1: Effect of survival threshold on score.

We ablated the survival threshold over $\{0.00, 0.25, 0.50\}$. The intermediate threshold achieved the highest score, suggesting that a mild survival constraint improves planning without being overly restrictive.

Time horizon	**Score**
5	0.14
10	0.75
15 (baseline)	0.89

Table 2: Effect of time horizon on score.

We ablated the time horizon over $\{5, 10, 15\}$. Performance decreased slightly from 15 to 10, and sharply from 10 to 5, suggesting that the task benefits from longer-horizon planning.

Sampling method	**Score**
Random	0.89
Stratified random (baseline)	0.89
Cross-entropy	0.86

Table 3: Effect of sampling method on score.

We ablated the action sequence sampling methods over random, stratified random, and cross-entropy methods (Rubinstein, 1999). Performance was similar across methods, suggesting that the choice of sampling method was not a major performance factor.

Sample count	Score
512	0.79
1024 (baseline)	0.89
2048	0.93

Table 4: Effect of sample count on score.

We ablated the action sequence sample count over $\{512, 1024, 2048\}$. Performance improved from 512 to 1024, with diminishing returns at 2048, suggesting that 1024 samples were fairly sufficient for effective planning on our task.

Architecture	Score
Ours (baseline)	0.89
DINO-WM	0.00

Table 5: Effect of architecture on score.

We tested our system architecture against one that borrows elements from DINO-WM (Zhou et al., 2025). The latter uses a DINOv2 ViT-S/14 architecture for the encoder, and a transformer architecture for the world model (Oquab et al., 2023). The DINO-WM architecture failed to complete the task. Because the encoder weights were frozen after ImageNet pre-training, we hypothesise that this failure can be attributed to limited task adaptation and the larger architecture requiring more data than was available.

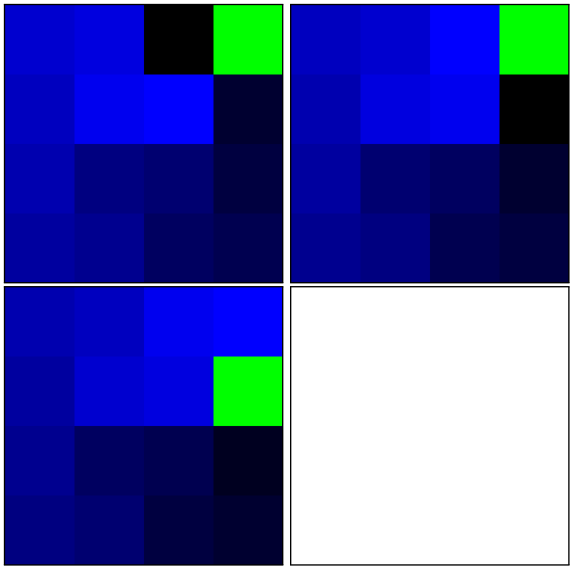


Figure 2: Snake win.

The snake was able to win when we sampled using the random method with $\tau = 0$. We hypothesise that this was because the final foods often require aggressive, nearly terminal manoeuvres. At high snake lengths, the legal path to the last food may simply pass through states that the survival head assigns low probability to, because such states are rare in the training data, or because small prediction errors are amplified when the board is almost full. When $\tau > 0$, the planner may thus discard these risky-looking trajectories in favour of safer actions that prolong the game without completing it.

System	Score
Random policy	0.00
Ours	0.89
Oracle planner	1.00
Hamiltonian cycle policy	1.00

Table 6: Comparison of systems by score.

We compared our system against a random policy, an oracle planner, and a Hamiltonian cycle policy. The random policy samples uniform randomly from $\mathcal{A}$ at each game tick. The oracle planner uses true underlying simulator state while planning, keeping the same settings as our system. The Hamiltonian cycle policy computes and follows a fixed Hamiltonian cycle throughout a game. The random policy achieved a score of 0.00, confirming that uninformed actions are insufficient even on a 4×4 grid. Both the oracle planner and Hamiltonian cycle policy achieved perfect completion. Our system achieved a score of 0.89, closing most of the gap between random play and the two perfect-information or hand-designed systems. This suggests that the learned world model captures enough game dynamics to support long-horizon planning.

5 Conclusion

We presented Snekboros, a vision-based world modelling approach for playing Snake. By encoding rendered game states into a learned representation, rolling those representations forward under candidate actions, and evaluating imagined trajectories, our system was able to achieve strong completion rates on a 4×4 Snake grid. Our ablations suggest that planning horizon and sample count play a larger role in performance than the precise sampling method, and that a learned task-specific encoder can substantially outperform a frozen pretrained visual encoder in this setting. Future work could explore having larger grids, stochastic food placement, and multiple snakes.

6 Appendix

Notation 1. Let $n \in \mathbb{N}_{\geq 1}$.

$$[n] = \mathbb{N}_{[1,n]}$$

Notation 2. Let $a, b \in \mathbb{N}_{\geq 1}$.

$$[a..b] = \mathbb{N}_{[a,b]}$$

Notation 3. Let $n \geq 1$, and X be a set, and $f_1, \ldots, f_n : X \to X$.

$$\bigcirc_{i=1}^{n} f_i = f_n \circ \cdots \circ f_1$$

References

Eloi Alonso, Adam Jelley, Vincent Micheli, Anssi Kanervisto, Amos Storkey, Tim Pearce, and François Fleuret. Diffusion for world modeling: Visual details matter in atari. In *Advances in Neural Information Processing Systems*, 2024. URL https://openreview.net/forum?id=NadTwTODgC.

Mahmoud Assran, Quentin Duval, Ishan Misra, Piotr Bojanowski, Pascal Vincent, Michael Rabbat, Yann LeCun, and Nicolas Ballas. Self-supervised learning from images with a joint-embedding predictive architecture, 2023. URL https://arxiv.org/abs/2301.08243.

Mido Assran, Adrien Bardes, David Fan, Quentin Garrido, Russell Howes, Mojtaba Komeili, Matthew Muckley, Ammar Rizvi, Claire Roberts, Koustuv Sinha, Artem Zholus, Sergio Arnaud, Abha Gejji, Ada Martin, Francois Robert Hogan, Daniel Dugas, Piotr Bojanowski, Vasil Khalidov, Patrick Labatut, Francisco Massa, Marc Szafraniec, Kapil Krishnakumar, Yong Li, Xiaodong Ma, Sarath Chandar, Franziska Meier, Yann LeCun, Michael Rabbat, and Nicolas Ballas. V-jepa 2: Self-supervised video models enable understanding, prediction and planning, 2025. URL https://arxiv.org/abs/2506.09985.

Randall Balestriero and Yann LeCun. Lejepa: Provable and scalable self-supervised learning without the heuristics, 2025. URL https://arxiv.org/abs/2511.08544.

Adrien Bardes, Quentin Garrido, Jean Ponce, Xinlei Chen, Michael Rabbat, Yann LeCun, Mahmoud Assran, and Nicolas Ballas. Revisiting feature prediction for learning visual representations from video, 2024. URL https://arxiv.org/abs/2404.08471.

Decart, Julian Quevedo, Quinn McIntyre, Spruce Campbell, Xinlei Chen, and Robert Wachen. Oasis: A universe in a transformer. 2024. URL https://oasis-model.github.io/.

Justin Fu, Aviral Kumar, Ofir Nachum, George Tucker, and Sergey Levine. D4rl: Datasets for deep data-driven reinforcement learning, 2021. URL https://arxiv.org/abs/2004.07219.

Danijar Hafner, Wilson Yan, and Timothy Lillicrap. Training agents inside of scalable world models, 2025. URL https://arxiv.org/abs/2509.24527.

K. Holkar, K. Wagh, and L. Waghmare. An overview of model predictive control. *International Journal of Control and Automation*, 3, 2011.

Po-Chen Ko, Jiayuan Mao, Yilun Du, Shao-Hua Sun, and Joshua B. Tenenbaum. Learning to act from actionless videos through dense correspondences, 2023. URL https://arxiv.org/abs/2310.08576.

Yann LeCun. A path towards autonomous machine intelligence, 2022. URL https://openreview.net/forum?id=BZ5a1r-kVsf.

Vincent Micheli, Eloi Alonso, and François Fleuret. Transformers are sample-efficient world models. In *International Conference on Learning Representations*, 2023. URL https://openreview.net/forum?id=vhFu1Acb0xb.

Maxime Oquab, Timothée Darcet, Théo Moutakanni, Huy Vo, Marc Szafraniec, Vasil Khalidov, Pierre Fernandez, Daniel Haziza, Francisco Massa, Alaaeldin El-Nouby, Mahmoud Assran, Nicolas Ballas, Wojciech Galuba, Russell Howes, Po-Yao Huang, Shang-Wen Li, Ishan Misra, Michael Rabbat, Vasu Sharma, Gabriel Synnaeve, Hu Xu, Hervé Jégou, Julien Mairal, Patrick Labatut, Armand Joulin, and Piotr Bojanowski. Dinov2: Learning robust visual features without supervision, 2023. URL https://arxiv.org/abs/2304.07193.

Reuven Y. Rubinstein. The cross-entropy method for combinatorial and continuous optimization. *Methodology and Computing in Applied Probability*, 1(2):127–190, 1999. doi: 10.1023/A:1010091220143.

Ilya Sutskever, Tim Salimans, and Durk Kingma. Requests for research 2.0. https://openai.com/index/requests-for-research-2/, January 2018. Accessed: 2026-04-27.

Richard S. Sutton. Dyna, an integrated architecture for learning, planning, and reacting. *SIGART Bulletin*, 2(4):160–163, 1991. doi: 10.1145/122344.122377. URL https://doi.org/10.1145/122344.122377.

John Tapsell. snake. https://johnflux.com/tag/snake/, 2015. Accessed: 2026-04-27.

Yuval Tassa, Yotam Doron, Alistair Muldal, Tom Erez, Yazhe Li, Diego de Las Casas, David Budden, Abbas Abdolmaleki, Josh Merel, Andrew Lefrancq, Timothy Lillicrap, and Martin Riedmiller. Deepmind control suite, 2018. URL https://arxiv.org/abs/1801.00690.

Wikipedia contributors. Snake (1998 video game). https://en.wikipedia.org/wiki/Snake_(1998_video_game), 2026. Accessed: 2026-04-27.

Grady Williams, Nolan Wagener, Brian Goldfain, Paul Drews, James M. Rehg, Byron Boots, and Evangelos A. Theodorou. Information theoretic mpc for model-based reinforcement learning. In *2017 IEEE International Conference on Robotics and Automation (ICRA)*, pages 1714–1721, 2017. doi: 10.1109/ICRA.2017.7989202. URL https://doi.org/10.1109/ICRA.2017.7989202.

Gaoyue Zhou, Hengkai Pan, Yann LeCun, and Lerrel Pinto. Dino-wm: World models on pre-trained visual features enable zero-shot planning, 2025. URL https://arxiv.org/abs/2411.04983.